Tema 5

Conversión y adaptación de documentos XML

Lenguaje de marcas y sistemas de gestión de la información

Tech Hub <FP>
euroformac

ÍNDICE

Objetivos de la unidad 3
1. Introducción 3
2. Técnicas de transformación de documentos XML 4
3. Lenguajes de transformaciones 6
4. Formatos de salida: HTML, XML, PDF, texto 8
4.1 Convertir XML en PDF utilizando XSL-FO y FOP 8
5. Descripción de la estructura y la sintaxis 9
4.1 Hojas de estilo XSLT 9
4.2 Enlazar documentos XML con hojas de estilo XSLT 12
6. Utilización de plantillas 13
5.1 <xsl:value-of> 13
5.2 <xsl:for-each> 15
5.2.1 Filtrado de contenido 17
5.2.2 Ordenando el contenido 17
5.2.3 Selección avanzada con <xsl:if> 18
5.2.4 Selección avanzada CHOOSE 18
6. Utilización de herramientas de procesamiento 21
6.1 DOM 21
6.2 SAX 23
6.3 Comparativa DOM vs SAX 24
6.4 Herramientas 25
7. Elaboración de documentación 28
8. Resumen 29
Glosario 30
Bibliografía 32

Objetivos de la unidad

- Convertir un documento XML de un formato a otro para facilitar su lectura o procesamiento en diferentes sistemas.
- Transformar datos de un esquema XML a otro para permitir la interoperabilidad entre diferentes sistemas de información.
- Adaptar un documento XML para que sea compatible con un software o plataforma específicos.
- Normalizar los datos en un conjunto de documentos XML para facilitar su integración y búsqueda.
- Simplificar la estructura y el contenido de los documentos XML para mejorar la eficiencia del procesamiento y la visualización en sistemas específicos.
- Mejorar la accesibilidad y la usabilidad de los documentos XML para diferentes usuarios y sistemas.
- Establecer un estándar común para la estructura y el contenido de los documentos XML en un conjunto de documentos para facilitar su uso y mantenimiento.

1. Introducción

XSL es un lenguaje de marcado que se utiliza para definir la presentación y la transformación de documentos XML. La sigla XSL significa **Extensible Stylesheet Language**, que se traduce como Lenguaje de Hojas de Estilo Extensibles. En términos básicos, XSL es un lenguaje que se utiliza para transformar y dar formato a la documentación en **formato XML**, lo que permite presentar los datos en una variedad de formas y estructuras que sean más legibles, accesibles y adecuadas para su uso en diferentes sistemas y dispositivos.

La transformación de documentos XML con XSL se logra mediante la definición de reglas de plantilla que se aplican al documento fuente a transformar, lo que permite la creación de múltiples versiones del mismo documento XML en diferentes formatos, desde documentos HTML para navegadores web hasta archivos de texto simples para procesadores de texto.

En resumen, XSL es un componente fundamental del procesamiento y la presentación de documentos XML en una variedad de sistemas y aplicaciones.

2. Técnicas de transformación de documentos XML

Definición

Designamos con la expresión latina "curriculum vitae" el documento físico o electrónico en el que presentamos nuestros datos personales, profesionales y académicos con el objeto de mostrar nuestra competencia para el puesto de trabajo ofertado y nuestro interés en participar en el proceso de selección.

Las técnicas de transformación de documentos XML se basa en:

- **Transformación:** Convirtiendo los ficheros XML en otro formato.
- **Formateo de objetos:** A través del diseño de un documento XML.
- **Búsqueda de contenido en documento:** A través del árbol de nodos de un documento XML.

El XSL es parte del XML y basa su versatilidad en tres **componentes fundamentales**:

- **XPath:** es un lenguaje para seleccionar o navegar entre los nodos en lenguaje XML dentro de un DOM.
- **XSL-FO:** FO de formateo de objetos (formating objects). Transformaciones para convertir de documento XML en otro tipo de información como HTML o PDF.
- **XSLT:** Lenguaje de transformación basado en hojas de estilo.
- **XQuery:** Lenguaje de consultas en documentos XML.

XSLT se utiliza para la transformación y es la parte más importante de XSL, por lo que se considera una recomendación del W3C.

El **objetivo de XSLT** es convertir los documentos XML en:

- Otros documentos XML.
- Otros formatos reconocibles por un navegador como HTML o XHTML, por ejemplo.

Generalmente, la transformación que hace XSLT es de uno a uno. Cada elemento XML se transformará en otro HTML o XHTML.

Una de las **ventajas** de las transformaciones es que pueden añadirse o quitarse elementos y atributos del fichero de salida. XSLT ofrece mucho juego, puesto que los elementos de un fichero pueden reestructurarse, ordenarse,

etc. Básicamente, una transformación XSLT consiste en modificar un árbol de entrada en uno de salida.

Una transformación XSLT se realiza usando un procesador XSLT, software que toma como entrada dos archivos (XML y XSL) y genera como documento de salida otro aplicando en la transformación la hoja de estilos XSL al fichero origen XML.

Transformación de XML + XSLT en otro XML

Los procesadores XSLT pueden ser integrados en un editor XML o en un explorador web o bien pueden invocarse desde la línea de comandos (como Xalan o Saxon).

Los procesadores XML son librerías que pueden estar integradas en cualquier tipo de programa.

Más info

Para ampliar sobre XSLT. Visualiza el video Introducción a XSLT

3. Lenguajes de transformaciones

XSLT 1.0 fue creada el 16 de noviembre de 1999 y, ocho años después, en el 2007, nació XSLT 2.0.

La versión 2.0 tiene bastantes mejoras con respecto a la versión 1.0.

Saxonica (Saxon) recomendó encarecidamente utilizar este estándar, que es compatible hacia delante (un programa que funcionaba con la versión 1.0 debería funcionar con procesadores con versiones superiores).

Actualmente, existen buenos procesadores XSLT como son Xalan, Saxon y MSXML.

Más info

Para saber más sobre los procesadores XSLT:

- Xalan
- Saxon
- MSXML

Entre las novedades de la versión 2.0: **[HACEMOS INTERACTIVIDAD]**

- Mejora del sistema de tipos.
- Mejora de las funciones incorporadas y los operadores.
- Mejora del modelo de datos.
- Soporte para expresiones regulares.
- Múltiples arboles como resultado.
- La instrucción <xsl:function>. Sirve para poder definir funciones que puedan ser utilizadas en expresiones XPath.
- Mejora del sistema de errores.
- Mejora de la funcionalidad de agrupación.
- Soporte de streaming, lo que permite procesar documentos muy grandes y generar resultados sin necesidad de cargarlos por completo en memoria.
- Nuevas instrucciones asociadas: <xsl:source-document>, <xsl:iterate>, <xsl:merge>, <xsl:fork>.
- Se añadió la modularidad a los programas creando el concepto de paquete como en Java.
- Un paquete define un interfaz que regula funciones, variables, plantillas y otros componentes que, pese a estar visibles fuera del paquete, pueden ser sobrescritas.

- El objetivo último es la reutilización y mantenibilidad del código.
- Otras instrucciones nuevas son:
  - **xsl:evaluate:** se utiliza para evaluar expresiones XPath que se construyen de manera dinámica con cadenas de caracteres.
  - **xsl:try:** este elemento se utiliza para recuperarse de errores que puedan ocurrir durante la ejecución de un programa.
  - **xsl:global-context-item:** se utiliza cuando se necesita hacer referencia a un elemento cuyo contexto es global en el documento.
  - **xsl:assert:** para ayudar a los desarrolladores a crear código robusto.

Tech Hub <FP> euroformac

4. Formatos de salida: HTML, XML, PDF, texto

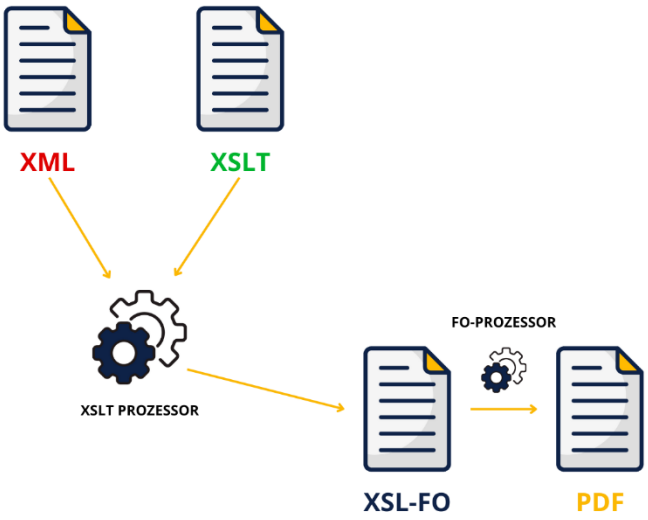
4.1 Convertir XML en PDF utilizando XSL-FO y FOP

Definición

Un **documento XSL-FO** se utiliza para especificar cómo se deben formatear los datos en un documento XML para su presentación en diferentes medios, ya sea en una pantalla, papel u otros formatos. Las siglas XSL-FO significan eXtensible Stylesheet Language Formatting Objects.

En un documento **XSL-FO**, se incluyen tanto los datos como el formato que se va a aplicar, y la unidad básica de trabajo es el "Formating Object", que se utiliza para presentar la información de manera formateada. Los objetos de formato pueden referirse a páginas, párrafos, tablas y otros elementos visuales que se utilizan para estructurar y presentar la información.

Al utilizar un documento XSL-FO, se pueden generar diferentes formatos de salida, como documentos PDF, RTF, PostScript, entre otros. Es importante reescribir el texto sin copiar literalmente la frase original y citar la fuente original en caso de utilizar información específica del mismo.



Proceso XSL-FO

5. Descripción de la estructura y la sintaxis

Transformaciones XSL (**XSLT**) es un lenguaje de programación declarativo que se utiliza para crear documentos a partir de archivos XML, tal como se muestra en la siguiente imagen:

Transformación documento XML

El archivo XML sirve como base para crear un resultado específico. El código fuente del programa que contiene las reglas de transformación que se aplicarán al archivo XML original se incluye en la hoja de estilo XSLT. El programa encargado de aplicar estas reglas de transformación al archivo XML inicial y producir el documento final deseado se conoce como procesador XSLT.

Se puede usar una nueva versión del archivo XML original o un documento en un formato diferente para este documento final. Se puede producir un documento utilizando XSLT a partir de un archivo XML, ya sea en XML o en otro formato. Una sola hoja de estilo XSLT se puede aplicar a varios archivos XML y se pueden usar diferentes hojas de estilo XSLT en un archivo XML para producir diferentes resultados.

4.1 Hojas de estilo XSLT

XSLT representa un **lenguaje declarativo** empleado para llevar a cabo la **transformación de documentos XML.** En contraste con otros lenguajes de programación, las hojas de estilo XSLT no se estructuran como una secuencia de instrucciones, sino más bien como una compilación de plantillas (template rules). Cada plantilla determina la forma en que se transforma un elemento específico, el cual se define mediante expresiones **XPath**.

El proceso de transformación de un documento XML mediante XSLT se desarrolla siguiendo los pasos que se detallan a continuación:

- En primer lugar, el procesador analiza el documento y crea una representación en forma de árbol.
- A continuación, el procesador recorre dicho árbol comenzando desde el nodo raíz.
- Durante este recorrido, el procesador decide si aplica alguna plantilla o no. En el caso de que un nodo no sea elegible para la aplicación de ninguna plantilla, su contenido se incorpora al documento final, es decir, el procesador incluye el texto del nodo en sí, no el de los nodos descendientes.
- Posteriormente, el procesador recorre los nodos hijos del nodo actual.

Por otro lado, cuando un nodo puede ser sometido a una plantilla, se aplica la plantilla correspondiente. Dicha plantilla puede generar texto que se incluirá en el documento final. En principio, el procesador no realiza un recorrido de los nodos hijos del nodo actual, a menos que la plantilla especifique explícitamente que dichos nodos hijos deben ser recorridos. Por último, una vez que el procesador ha concluido el recorrido del árbol, se considera que la transformación del documento ha finalizado.

En una hoja de estilo XSLT, se incluyen las etiquetas necesarias para llevar a cabo la transformación del documento XML, como por ejemplo la declaración de los **namespaces**, las plantillas y las expresiones XPath que se utilizan para seleccionar los elementos a transformar.

Un documento de hoja de estilo XSLT es un archivo XML que tiene ciertas etiquetas esenciales que permiten definir las reglas para la transformación de otro documento XML. Algunas de las etiquetas mínimas necesarias son las siguientes:

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet
xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
version="1.0"> </xsl:stylesheet>
```

Dentro del contexto de una hoja de estilo XSLT, es necesario incorporar ciertas etiquetas esenciales para posibilitar la interpretación del archivo por parte del procesador XML. Dichas etiquetas comprenden:

- La declaración XML **<?xml>**, la cual se encuentra presente en todo documento XML.

- La instrucción **xsl:stylesheet**, desempeñando el papel de etiqueta principal en la hoja de estilo. Sus atributos especifican la versión y el espacio de nombres correspondiente.

Dentro de la instrucción xsl:stylesheet, es posible incluir elementos de alto nivel, así como plantillas que definen las reglas de transformación.

Por ejemplo:

```
<?xml version="1.0" encoding="UTF-8"?> <xsl:stylesheet
xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
version="1.0">
   <xsl:output method="xml" indent="yes"/>
   <xsl:template match="/">
   </xsl:template>
</xsl:stylesheet>
```

Estas etiquetas son fundamentales en una hoja de estilo XSLT:

- La declaración **xml <?xml>**, presente en todo documento XML.
- La etiqueta raíz de la hoja de estilo en el lenguaje de marcas es **xsl:stylesheet**, y sus atributos definen la versión y el espacio de nombres correspondiente. Dentro de esta etiqueta se encuentran elementos de alto nivel y plantillas, tal como se ilustra en el siguiente ejemplo:
  - El elemento de alto nivel **xsl:output** indica el tipo de salida generada.
  - La instrucción **xsl:template** define una plantilla. El atributo match especifica los elementos afectados por dicha plantilla y contiene una expresión XPath. El contenido de la instrucción define la transformación que se aplicará. En el caso de que la instrucción esté vacía, como en el ejemplo previo, el nodo será reemplazado por nada, es decir, se eliminará, mientras que el texto contenido en el elemento se mantendrá.
- Por defecto, al aplicar una plantilla a un nodo, no se recorren sus nodos descendientes. Para indicar que se desea recorrer los nodos descendientes y aplicarles las plantillas correspondientes, se utiliza la instrucción **xsl:apply-templates**.
  Por ejemplo:

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet
xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
version="1.0">
   <xsl:template match="/">
```

```
        <xsl:apply-templates />
    </xsl:template>
    <xsl:template match="elemento">
    </xsl:template>
</xsl:stylesheet>
```

4.2 Enlazar documentos XML con hojas de estilo XSLT

La hoja de estilo XSLT puede ser asociada de manera permanente a un documento XML mediante la instrucción de procesamiento "`<?xml-stylesheet ?>`", que también se utiliza para asociar hojas de estilo CSS. Dicha instrucción debe ser ubicada al inicio del documento, después de la declaración XML.

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="ejemplo.xsl"?>
```

Al visualizar un documento XML vinculado con una hoja de estilo XSLT en un navegador web, el resultado de la transformación se muestra en la página. Sin embargo, si se muestra el código fuente de la página, se mostrará el documento XML original.

- En **Firefox** y **Microsoft Edge**, al abrir un documento XML vinculado con una hoja de estilo XSLT, siempre se mostrará el documento transformado.

- Pero en **Google Chrome**, solo se mostrará el documento transformado si se accede a través de un servidor web (http://...). Si se abre directamente como un archivo local (file://...), Google Chrome mostrará una página en blanco. Además, en Google Chrome, el documento transformado solo se mostrará si está bien formado, mientras que en Firefox y Microsoft Edge, se mostrará el resultado de la transformación independientemente de si está bien formado o no.

Más info

Para ampliar información sobre cómo crear un documento XSLT visualiza el video XSLT y HTML

6. Utilización de plantillas

La operativa básica para transformar archivos XML no es complicada, pero la creación de la plantilla correcta para realizar una transformación concreta puede ser más o menos costosa, para lo cual, XSLT tiene una serie de etiquetas.

5.1 <xsl:value-of>

Esta etiqueta extrae el valor de un elemento XML y lo añade al flujo de salida de la transformación.

Con el atributo **select**, el cual utiliza una expresión XPath, se seleccionará el elemento concreto.

Como se ha mencionado anteriormente, XPath es un lenguaje que permite la selección de secciones específicas de un documento XML y la recuperación de la información deseada (nodo de contexto). Teniendo el siguiente documento .xml.

```
<?xml version="1.0" encoding='ISO-8859-1'?>
<?xml-stylesheet href="tienda.xsl" type="text/xsl"?>
<tienda>
    <nombre>La tiendecilla</nombre>
    <telefono>953 87 12 23</telefono>
    <producto>
       <codigo>92</codigo>
       <cantidad>10</cantidad>
       <articulo>Radio-Casette</articulo>
    </producto>
    <producto>
       <codigo>103</codigo>
       <cantidad>50</cantidad>
       <articulo>Reloj Cocina</articulo>
       <marca>Casio</marca>

    </producto>
    <producto>
       <codigo>1312</codigo>
       <cantidad>3</cantidad>
```

```
        <articulo>Sofá</articulo>
      </producto>
    <producto>
        <codigo>104</codigo>
        <cantidad>25</cantidad>
        <articulo>Reloj Salón</articulo>
        <marca>Casio</marca>
    </producto>
    <producto>
        <codigo>105</codigo>
        <cantidad>40</cantidad>
        <articulo>Despertador</articulo>
        <marca>Casio</marca>
    </producto>
</tienda>
```

Por ejemplo, para mostrar el valor del elemento artículo, que es un hijo del elemento producto, podríamos utilizar la siguiente regla:

```
<xsl:template match="//producto">
   <xsl:value-of select="./articulo"/>
</xsl:template>
```

El atributo "select" se utiliza para especificar una expresión XPath que indica la ubicación de los datos en el documento XML que se desea seleccionar. Por ejemplo, si se tiene un elemento "artículo" que es hijo del elemento actual (en este caso, el elemento "producto"), se puede seleccionar el valor del elemento "artículo" con la expresión "./artículo".

Esto se indica mediante:

Expresión	Expresion XPath
nodename	Selecciona todos los nodos secundarios del nodo denominado
/	Selecciona desde el nodo raíz
//	Selecciona nodos en el documento desde el nodo actual que coincidan con la selección sin importar donde se encuentran
.	Selecciona el nodo actual
..	Selecciona el padre del nodo actual

@	Selecciona los atributos

5.2 <xsl:for-each>

En esta etiqueta **<xsl:for-each>**, se seleccionarán todos los elementos uno a uno de una lista determinada.

Los elementos que van a utilizarse estarán filtrados por el atributo **select**, el cual utilizará una expresión XPath.

Más info

Para ampliar información sobre las plantillas en XSL visualiza el video Plantillas en XSLT.

https://www.youtube.com/watch?v=jAqHyn_G5rQ

En XPath para los nodos que pertenecen a un conjunto de nodos las funciones posicionales permiten buscar coincidencias en la posición de estos nodos dentro del conjunto:

Función	Ejemplo	Descripción
position ()	`<xsl:template match="/libro/capitulo[position()=2]"/>`	Devuelve la posición del nodo dentro de un conjunto de nodos. En el ejemplo buscamos la información del nodo hijo de capitulo que ocupa

		la segunda posición.
last()	`<xsl:template match="/libro/capitulo[position()=last()]"/>`	Devuelve el nodo que ocupa la última posición en un conjunto de nodos.
count()	`<xsl:template match="count(nodo)"/>`	Devuelve el número de nodos dentro del conjunto de nodos

Los predicados son expresiones que van entre los signos [].

Por ejemplo:

Ejemplos	Descripción
capitulo[1]	Selecciona el primer nodo capitulo hijo del nodo actual
capitulo[position()=1]	Selecciona el primer nodo capitulo hijo del nodo actual
capitulo[last()]	Selecciona el ultimo nodo capitulo hijos del nodo actual
capitulo[@titulo=" Mi casa"]	Selecciona el nodo capitulo que tiene como valor del atributo título " Mi casa"

5.2.1 Filtrado de contenido

Cuando un fichero es muy grande, lo más normal es filtrar el contenido y no trabajar con todo el árbol de nodos.

La etiqueta `<xsl:for-each>` podría modificarse de la siguiente forma para que filtre solamente los artículos de la marca Casio:

```
<xsl:for-each match="tienda/producto[marca='Casio']">
   <tr>
      <td><xsl:value-of select="codigo"/></td>
      <td><xsl:value-of select="cantidad"/></td>
      <td><xsl:value-of select="articulo"/></td>
   </tr>
</xsl:for-each>
```

En este caso, el procesador recorrerá solo los nodos <producto> que tengan un elemento <marca> con el valor Casio, ignorando el resto.

5.2.2 Ordenando el contenido

En este momento se necesita que, además de la marca Casio, los productos queden ordenados por la cantidad de artículos pedidos. Es preciso utilizar la etiqueta <xsl:sort> añadiendo la propiedad select=" cantidad" .

El resultado sería añadir la siguiente línea al fichero:

```
<xsl:sort select="cantidad"/>
```

El filtro incluyendo la ordenación:

```
<xsl:for-each match="tienda/producto[marca='Casio']">
   <xsl:sort select="cantidad"/>
   <tr>
      <td><xsl:value-of select="codigo"/></td>
      <td><xsl:value-of select="cantidad"/></td>
      <td><xsl:value-of select="articulo"/></td>
   </tr>
</xsl:for-each>
```

Importante

Se sobreentiende que los campos que se ordenan tendrán que ir en la salida. No tiene mucho sentido ordenar por un campo que, posteriormente, no va a aparecer en la salida.

5.2.3 Selección avanzada con <xsl:if>

Si quieren mostrarse todos los artículos cuya cantidad sea superior a 25 unidades. Para esta operación, puede utilizarse la etiqueta <xsl:if> de la siguiente forma:

```
<xsl:if test="cantidad>25"/>
```

El lugar donde hay que colocar esta sentencia es siempre dentro de la etiqueta `<xsl:for-each>`

Ejemplo: Filtrar los productos de la marca Casio que tengan más de 25 unidades, y además ordenarlos por cantidad:

```
<xsl:for-each match="tienda/producto[marca='Casio']">
   <xsl:if test="cantidad>25"/>
   <xsl:sort select="cantidad"/>
   <tr>
      <td><xsl:value-of select="codigo"/></td>
      <td><xsl:value-of select="cantidad"/></td>
      <td><xsl:value-of select="articulo"/></td>
   </tr>
</xsl:for-each>
```

Importante

El contenido del atributo test en <xsl:if> siempre debe ser una expresión XPath.

5.2.4 Selección avanzada CHOOSE

Se necesita conseguir el resultado siguiente:

Artículos			
Código	**Artículo**	**Marca**	**Cantidad**
92	Radio-casette		10
103	Reloj cocina	Casio	50

1312	Sofá		3
104	Reloj salón	Casio	25
105	Despertador	Casio	40

Dependiendo de la cantidad de cada artículo, la cantidad se sombreará de amarillo si supera el valor de 25 y de verde en caso contrario.

```
<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
<xsl:template match="/">
<html>
<body>
   <h3>ARTICULOS</h3>
   <table border="1">
      <tr bgcolor="#CFFFFFF">
         <th style="text-align:left">CODIGO</th>
         <th style="text-align:left">ARTICULO</th>
         <th style="text-align:left">MARCA</th>
         <th style="text-align:left">CANTIDAD</th>
      </tr>
      <xsl:for-each select="tienda/producto">
      <tr>
         <td><xsl:value-of select="codigo"/></td>
         <td><xsl:value-of select="articulo"/></td>
         <td><xsl:value-of select="marca"/></td>
           <xsl:choose>
              <xsl:when test="cantidad>25">
                 <td bgcolor="#ffff66"><xsl:value-of
select="cantidad"/></td>
              </xsl:when>
              <xsl:otherwise>
                 <td bgcolor="#99ff99"><xsl:value-of
select="cantidad"/></td>
              </xsl:otherwise>
           </xsl:choose>
      </tr>
      </xsl:for-each>
   </table>
</body>
</html>
```

```
</xsl:template>
</xsl:stylesheet>
```

Tech Hub <FP> euroformac

6. Utilización de herramientas de procesamiento

Existen diversas herramientas para analizar documentos XML y definir su estructura, entre las cuales se encuentran **DOM** y **SAX**. Además de estas, hay otras herramientas disponibles.

Podemos clasificar estas herramientas en **validantes** y **no validantes**.

Las **herramientas validantes** se encargan de comprobar la correcta estructura y conformidad de un documento con las reglas de XML, de acuerdo a una Definición del Tipo de Documento (DTD) establecida.

Los analizadores DOM y SAX son independientes del lenguaje de programación y cuentan con versiones específicas para diversos lenguajes, tales como Java, VisualBasic, C, entre otros.

6.1 DOM

Definición

El **Document Object Model** (**DOM**) es un modelo estandarizado de objetos utilizado para representar documentos HTML y XML. Se compone de un conjunto de interfaces que describen la estructura abstracta de un documento XML

Las **herramientas basadas en DOM** permiten insertar, eliminar y reorganizar los nodos del documento XML mediante llamadas a funciones y procedimientos que permiten acceder, cambiar, borrar o añadir nodos de información en los documentos XML.

DOM es una **API** para documentos **HTML** y **XML** que define la estructura lógica de los documentos y la manera en que se accede y manipula el documento. Aunque el término "documento" se refiere tradicionalmente a una estructura textual, el XML se utiliza para representar muchos tipos de información que se almacenan en diferentes sistemas, y el DOM puede manipular estos datos.

El uso de DOM permite a los programadores:

- Construir documentos
- Navegar por su estructura
- Añadir, modificar o eliminar elementos y contenido.

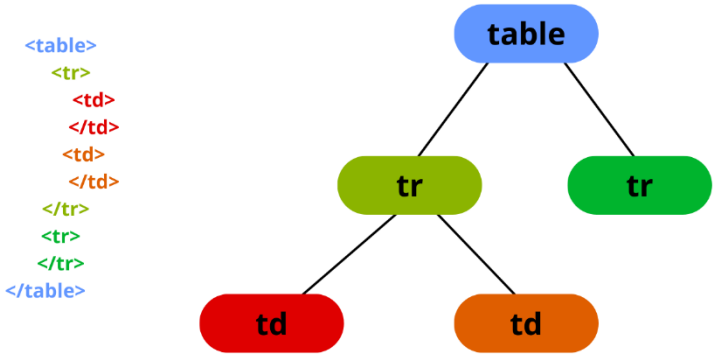
La API se puede utilizar en cualquier lenguaje de programación, como **Java** o **ECMAScript**, y proporciona una interfaz estándar de programación que puede utilizarse en una amplia variedad de entornos y aplicaciones.

Importante

Es importante tener en cuenta que al hablar de la interfaz de DOM (o SAX), no nos referimos a una interfaz gráfica, sino a una interfaz de aplicaciones.

Se trata de un dispositivo que posibilita la comunicación entre dos sistemas que emplean lenguajes distintos. La API se encarga de establecer el diálogo con los datos, permitiendo el acceso y la manipulación de los mismos.

Existen otras herramientas, aparte de DOM y SAX, que se utilizan para analizar el lenguaje XML y definir la estructura de un documento. Podemos distinguir entre **herramientas validantes** y **no validantes**. Las primeras se encargan de verificar que el documento, además de cumplir con las reglas de XML en cuanto a su formato, se ajuste a una estructura definida en una Definición del Tipo de Documento (DTD). Los analizadores DOM y SAX son independientes del lenguaje de programación y cuentan con versiones específicas para diferentes lenguajes de programación.



DOM

6.2 SAX

SAX (**Simple API for XML**) es una interfaz que proporciona un acceso sencillo e intuitivo a archivos XML. Se utiliza en situaciones en las que los archivos XML tienen una estructura similar a la que se desea obtener, principalmente cuando los datos almacenados en los documentos XML son generados o legibles por máquina. Cuando los datos son de este tipo, es común crear una estructura de datos y clases que permitan: su ordenamiento**,** manipulación**,** almacenamiento.

SAX posibilita la creación ágil de una herramienta que permita generar instancias de modelos de objetos fundamentados en el almacenamiento de datos de los documentos XML.

SAX se basa en un mecanismo de lectura **event-driven (dirigido por eventos).** Cada vez que el analizador encuentra un elemento, atributo, texto o final de nodo, se dispara un evento que puede ser capturado y tratado mediante funciones específicas del programa.

Esto permite, por ejemplo, que un programador o documentalista lea un XML con una lista de contactos y cree, a partir de él, un modelo de objetos con clases que representen cada persona y sus atributos.

Ejemplo ilustrativo

Un documento XML puede contener elementos de persona, con información como nombre y correo electrónico.

- El modelo de objetos generado con SAX podría incluir una clase LibretaDeDirecciones, que actúe como contenedor de múltiples Personas.
- Cada clase Persona almacenaría los datos de nombre y correo electrónico.
- El procesador SAX sería el encargado de identificar cada elemento del XML y colocarlo en el objeto adecuado, construyendo así una estructura en memoria que puede manipularse y ordenarse según las necesidades.

Ventajas principales de SAX:

- Bajo consumo de memoria, ya que no se carga el documento completo.
- Ideal para procesar documentos XML extensos.
- Lectura rápida gracias al procesamiento secuencial.

Inconveniente:

No permite acceder de forma aleatoria a los nodos (como sí hace DOM), lo que puede dificultar ciertas operaciones de manipulación

6.3 Comparativa DOM vs SAX

	DOM	SAX
Tipo de API	API basada en árbol	API basada en eventos
Procesamiento	Carga todo el documento en memoria y construye un árbol de objetos	Analiza el documento en orden secuencial y dispara eventos cuando encuentra elementos
Memoria	Requiere suficiente memoria para contener todo el documento	Requiere muy poca memoria, ya que solo mantiene en memoria el elemento actual que está siendo procesado
Velocidad	Más lento, especialmente con documentos grandes	Más rápido, especialmente con documentos grandes
Capacidad de modificación	Permite modificar el documento en memoria y luego escribir los cambios de vuelta al archivo original	No permite modificar el documento, solo leerlo
Manejo de errores	Detecta errores durante la carga del documento	Detecta errores durante el procesamiento del documento
Uso recomendado	Documentos pequeños o medianos	Documentos grandes o cuando se requiere un procesamiento más rápido y no se necesita modificar el documento

6.4 Herramientas

Las herramientas más empleadas son:

- **Oxygen XML Editor**

Es uno de los más utilizados por los desarrolladores XSLT.

Pueden realizarse todas las acciones necesarias con los documentos XSLT, como puede ser su creación, edición, testeo o validación.

Más info

Puedes descargarte la aplicación en:

https://www.oxygenxml.com/xml_editor/xslt_editor.html

- **Altova Editor XSLT**

Tiene todos los aspectos de un editor avanzado como el coloreado de sintaxis, marcadores, finalización de código, etc.

Asimismo, tiene funciones de herramientas avanzadas como la generación y evaluación de XPath, transformación de alta velocidad y explorador web integrado.

Más info

Puedes descargarte la aplicación en:

https://www.altova.com/es/xmlspy-xml-editor/xslt-editor

- **Xpontus XML Editor**

Es un editor XML opensource no muy sofisticado, pero que tiene muchas funciones que pueden ayudar a un programador a realizar un pequeño proyecto con XML.

Más info

Puedes descargarte la aplicación en:

http://xpontus.sourceforge.net/

- **Herramientas online**

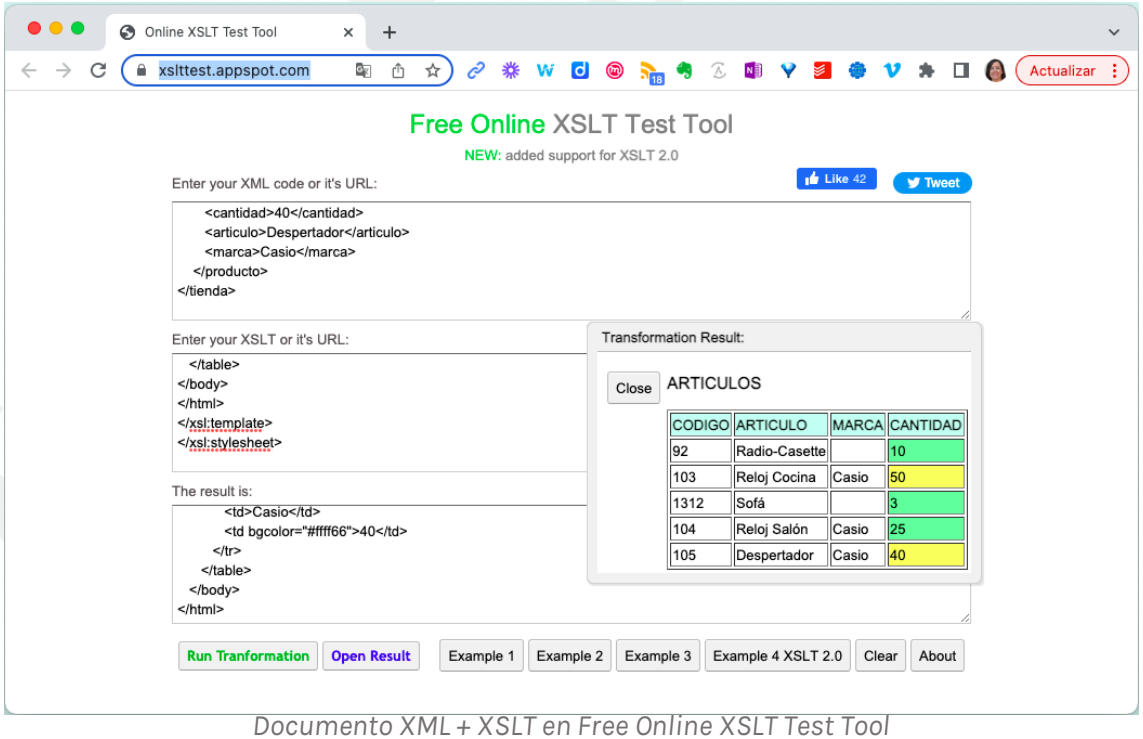
Se recomienda utilizar XSL Test Online, que puede servir para hacer transformaciones en línea desde cualquier dispositivo.

Estas herramientas tienen la ventaja de poder ejecutarse desde prácticamente cualquier entorno, pero no ofrecen las mismas funcionalidades que una herramienta de escritorio como Altova u Oxygen u las vistas desde el inicio de este módulo.

Más info

Tienes acceso en:

Free Online XSLT Test Tool

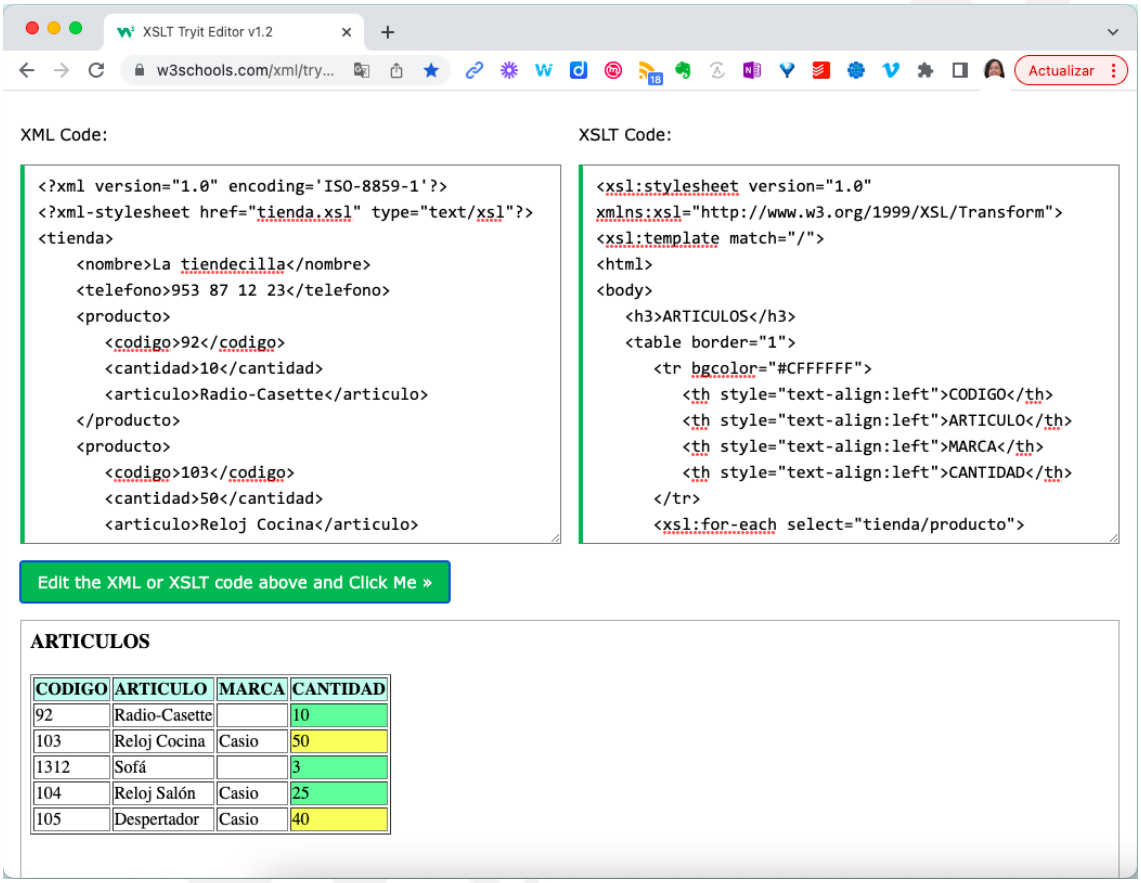


Documento XML + XSLT en Free Online XSLT Test Tool

Más info

Tienes acceso en:

[XSLT Tryit Editor v1.2]



Documento XML + XSLT en XSLT Tryit Editor v1.2 (Fuente: elaboración propia)

7. Elaboración de documentación

En diversos entornos de programación, como Visual Basic, se puede generar de manera sencilla documentación XML para proyectos. Esta herramienta puede crear automáticamente un esqueleto XML para tipos y miembros, y luego se pueden agregar resúmenes, descripciones y comentarios para cada parámetro.

Al configurar adecuadamente, la documentación XML se genera automáticamente en un archivo con el mismo nombre que el proyecto. La herramienta de documentación **Oxygen XML Schema**, también permite generar fácilmente documentación completa para los modelos de contenido de XML Schema, incluidos diagramas completos, anotaciones y referencias cruzadas.

Es posible generar documentación de esquema XML en varios formatos, incluidos HTML, PDF y DocBook.

Se pueden ver algunos ejemplos de documentación generada por Oxygen:

- Ejemplo en HTML
- Ejemplo en PDF

8. Resumen

Los **archivos XML** son una herramienta popular para organizar la información de manera jerárquica, lo que los hace fácilmente legibles. Sin embargo, para darle un valor añadido a esta información o hacerla comprensible para cualquier persona, es necesario procesarla. Es aquí donde entra en juego **XSLT**, un estándar para transformar estos documentos XML. La importancia de XSLT se refleja en el hecho de que ha evolucionado a su versión 3.0, lo que significa que ha sido adoptado por una gran cantidad de desarrolladores y empresas.

La capacidad de transformar archivos XML en diferentes formatos, como HTML o PDF, ha hecho que XSLT sea una herramienta esencial en la creación de documentos y sitios web. Además, su capacidad para generar informes y visualizaciones de datos ha ayudado a las empresas a tomar decisiones basadas en datos más informadas.

En resumen, la importancia de XSLT radica en su capacidad para transformar y procesar información de manera efectiva y eficiente, lo que ha llevado a su amplia adopción en el desarrollo de software y en la creación de sitios web.

Glosario

- **XSLT:** (Extensible Stylesheet Language Transformations) es un lenguaje de marcas utilizado para transformar documentos XML en otros formatos, como HTML, PDF o XML mismo. Es parte del conjunto de tecnologías XML y se basa en patrones y reglas predefinidas para realizar la transformación.

- **DOM:** Document Object Model (Modelo de Objetos de Documento), es una interfaz de programación que proporciona una representación estructurada y jerárquica de documentos HTML, XML o XHTML. Es una especificación independiente del lenguaje de programación y define la forma en que los documentos se organizan y se accede a ellos mediante la creación de una estructura de árbol de objetos. El DOM permite manipular dinámicamente los elementos de un documento mediante la adición, modificación o eliminación de nodos y atributos. Proporciona un conjunto de métodos y propiedades que permiten a los desarrolladores acceder, navegar y modificar el contenido, la estructura y el estilo de un documento.

- **SAX:** (Simple API for XML) es una interfaz estándar utilizada en el desarrollo de aplicaciones que trabajan con documentos XML. Se trata de un enfoque basado en eventos para el análisis y procesamiento de archivos XML. A diferencia del enfoque de modelo de árbol, donde el documento XML se carga completamente en memoria como una estructura de datos, SAX adopta un enfoque más ligero y eficiente al procesar el archivo XML de forma secuencial.

- **Xpath:** Se utiliza para la navegación y extracción de información de documentos XML. Proporciona una forma de seleccionar elementos específicos dentro de un documento XML utilizando rutas o expresiones que describen la ubicación de los nodos en la estructura del documento. Con XPath, se pueden realizar consultas y filtrar datos en un documento XML, accediendo a elementos individuales, conjuntos de elementos o atributos. Utiliza una sintaxis sencilla y compacta para especificar los caminos y condiciones de búsqueda, lo que facilita la manipulación y extracción de información precisa.

- **XSL-FO:** (Extensible Stylesheet Language Formatting Objects) es un lenguaje de marcas utilizado para describir el formato visual y la presentación de documentos XML. Es una especificación del World Wide Web Consortium (W3C) que proporciona un conjunto de

elementos y propiedades para definir cómo se debe presentar un documento XML cuando se transforma en un formato de salida final, como PDF, impresión o visualización en pantalla. XSL-FO permite definir reglas de formato, diseño y estructura para elementos individuales y grupos de elementos en un documento XML. Utiliza una sintaxis basada en etiquetas y atributos para establecer propiedades como tamaño y tipo de fuente, alineación de texto, márgenes, espaciado, columnas, tablas y otros aspectos visuales.

- **Xquery:** Es un lenguaje de consultas utilizado para recuperar y manipular información almacenada en documentos XML y otros recursos de datos estructurados. Es un estándar de consulta desarrollado por el Consorcio World Wide Web (W3C) y se utiliza ampliamente en sistemas de gestión de bases de datos XML y en entornos de desarrollo web. XQuery proporciona una sintaxis expresiva y poderosa para realizar consultas y transformaciones en datos XML. Permite buscar datos específicos dentro de documentos XML utilizando expresiones y condiciones flexibles, lo que facilita la extracción precisa de información deseada. Además de las consultas básicas, XQuery admite operaciones avanzadas como filtrado, ordenamiento, agrupación y unión de datos de diferentes fuentes.

Bibliografía

- Zurdo, S. F. J. T. R. & RA-MA Editorial. (2023). *Lenguajes de Marcas y Sistemas de Gestión de la Información. Cfgs.* (1.ª ed.). RA-MA Editorial.
- Castro, J. M., & Rodríguez, J. R. (2001). *Lenguajes de marcas y sistemas de gestión de la información*. Garceta.
- *XSLT Tutorial*. (s. f.). http://www.w3bai.com/es/xsl/default.html
- *XPath: XML Path language. XML. Bartolomé Sintes Marco. www.mclibre.org.* (s. f.). https://www.mclibre.org/consultar/xml/lecciones/xml-xpath.html
- *Tema 2: XSLT.* (s. f.). http://www.jtech.ua.es/j2ee/2003-2004/modulos/xml/sesion02-apuntes.htm